Module 6: FSDP and ZeRO

Trading Redundant Memory for Collective Communication

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. Where is the redundancy?
2. Execution and wrapping
3. Choosing and comparing approaches

Guiding question

What work and state does DDP repeat, and how could devices cooperate to reduce that redundancy?

What should be clear by the end?

1. Identify repeated optimizer work and state in DDP.
2. Develop a way to share responsibility for the update across devices.
3. Trace the resulting state through forward and backward.
4. Reason about memory, communication, and implementation trade-offs.

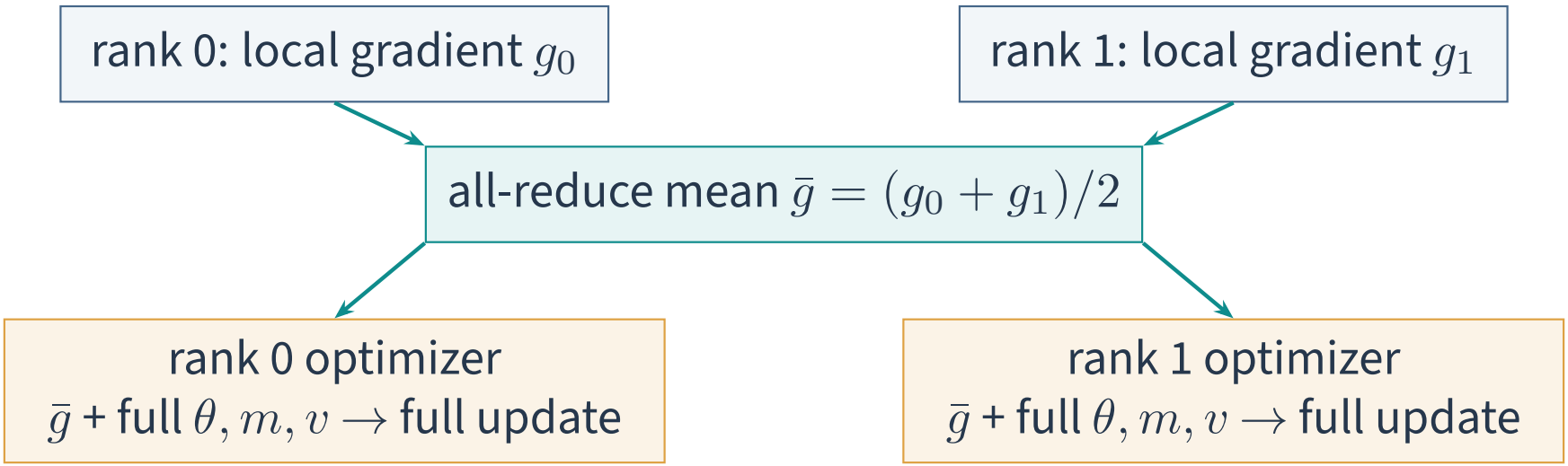
PART 1

Where is the redundancy?

---

1. Where is the redundancy?
2. Execution and wrapping
3. Choosing and comparing approaches

After synchronizing, what does each optimizer do?



Same synchronized gradient + same starting weights and optimizer state.

But wait! Is there some redundancy here?

Both ranks store the full optimizer state and repeat the same update. Can they share this work? What must each rank have for the next forward pass?

Explore: can the ranks share the update?

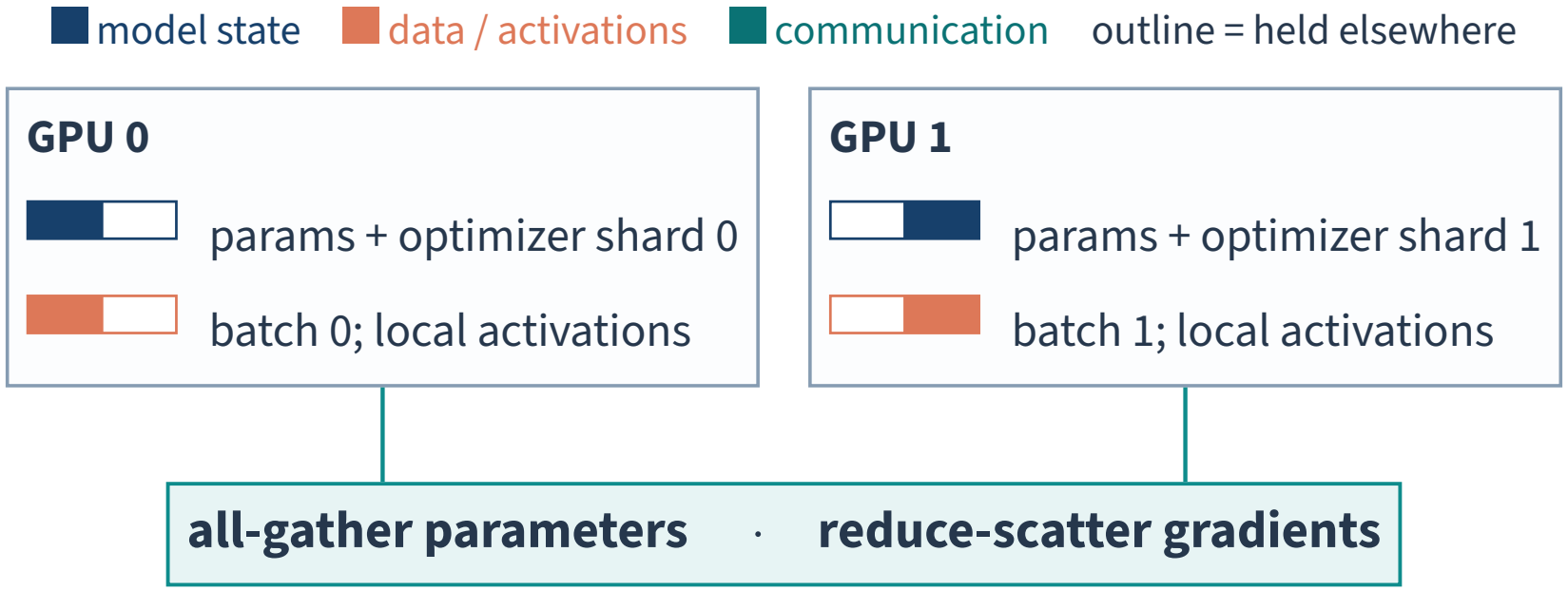
Interactive comparison

Open the sharded-training explorer ↗

1. Follow the DDP update: what does each rank repeat?
2. Split the optimizer state: which parameters does each rank update? How does the other rank get them?
3. Shard the model too: what must be gathered to compute? Where do reduced gradient shards go?

Step through the same training phases in each strategy. The explorer is a two-rank teaching model; actual FSDP resharding and overlap depend on its configuration.

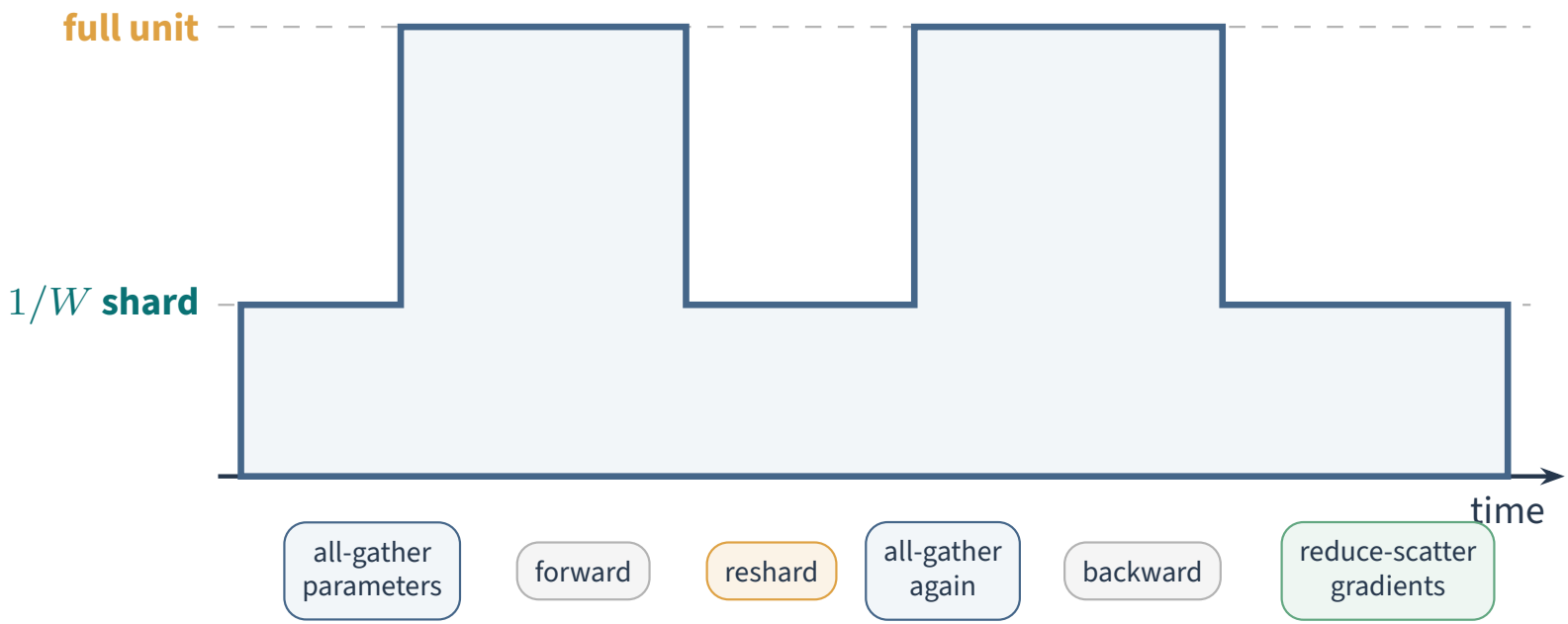
Parallelism map: FSDP shards model state



What changes from DDP? Model state is sharded. Each device gathers the active unit's full parameters to compute its batch.

Full-shard example. Gradient shards persist after reduction; details depend on FSDP configuration.

Sharding changes when full parameters exist



Full parameters are transient around compute; the local shard is persistent.

Now account for what we saw

From the visualization to a memory budget

Which state stayed replicated? What was persistent, and what was materialized only for computation?

The next slides summarize the layouts and their assumptions.

DDP replicates persistent state on every rank

State	bytes / param.	DDP placement
BF16 parameters	2	replicated
BF16 gradients	2	replicated
FP32 master copy	4	replicated
FP32 Adam moment m	4	replicated
FP32 Adam moment v	4	replicated
total model state	about 16	on every GPU

Example configuration; other precision and optimizer choices use different storage.

The scaling limit

More GPUs can improve throughput but do not reduce DDP's replicated state.

ZeRO removes one class of redundancy at a time

Method	optimizer state	gradients	parameters
DDP	replicated	replicated	replicated
ZeRO-1	sharded	replicated	replicated
ZeRO-2	sharded	sharded	replicated
ZeRO-3 / full-shard FSDP	sharded	sharded	sharded

Idealized per-rank model state

With full sharding and world size W, persistent model-state memory approaches $1/W$ of the unsharded total, before activations, buffers, and temporary materialization.

Source: Hugging Face Ultra-Scale Playbook and ZeRO

What changes at each ZeRO stage?

Stage	Training-step consequence
ZeRO-1	Each rank updates its owned parameters; updated shards are shared so replicas can compute the next forward.
ZeRO-2	Reduce-scatter leaves each rank its reduced gradient shard for that update.
ZeRO-3	Parameters are sharded too: all-gather the active unit before local computation.

Still data parallel

Each rank processes its own batch; sharding divides model-state ownership, not the batch's sequence of layers.

A memory budget: persistent is not peak

Example: 1B parameters, four ranks, 16 bytes / parameter

DDP: 16 GB per rank $\longrightarrow$ ideal full sharding: $16/4 = 4$ GB per rank

Decimal GB; the subtotal assumes the preceding precision and optimizer layout.

What else occupies the peak?

Temporary full units and prefetch, activations, collective buffers, and allocator overhead. The local shard is already counted: add only the extra materialized storage.

Two collectives make sharding usable

	Rank 0	Rank 1
Start: parameter shards	P_0	P_1
All-gather parameters	$[P_0, P_1]$	$[P_0, P_1]$
Local forward / backward	gradient $[g_{00}, g_{01}]$	gradient $[g_{10}, g_{11}]$
Reduce-scatter gradients	$g_{00} + g_{10}$	$g_{01} + g_{11}$

The division of work

Each rank gathers the full unit to compute its own data; after reduction it retains the gradient for its owned shard. Averaging requires the appropriate world-size normalization.

Keep or reshard after forward?

Keep full parameters

Gather → forward → backward

More residency; avoids a backward gather.

Reshard after forward

Gather → forward → reshard → gather → backward

Less residency between uses; another gather.

The explorer uses the first policy; the preceding lifecycle shows the second. Exact scheduling depends on configuration.

PART 2

Execution and wrapping

1. Where is the redundancy?
2. Execution and wrapping
3. Choosing and comparing approaches

One FSDP unit is a materialization boundary

Large unit

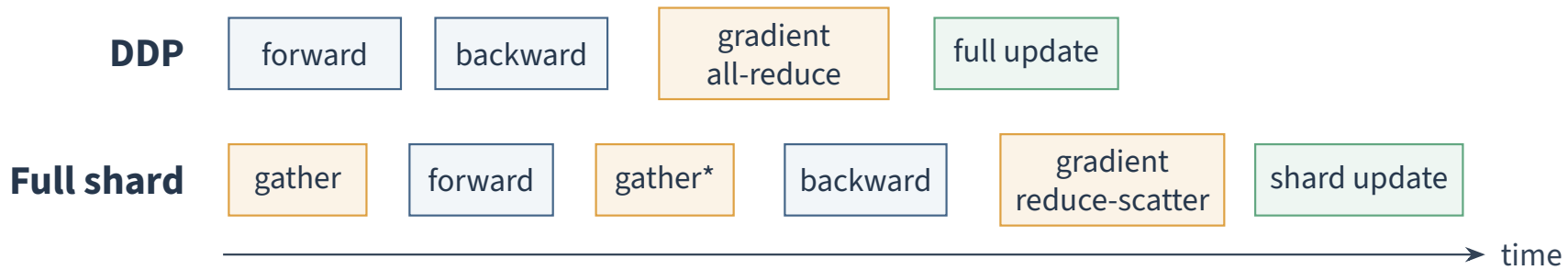
fewer collectives
larger peak all-gather
less opportunity for overlap
coarser memory release

Tiny units

low peak materialization
many latency-dominated collectives
more metadata and scheduling overhead

Wrapping each Transformer block is a common starting point because it aligns materialization with a natural compute unit.

Sharding buys memory with communication



*If parameters were resharded after forward. This schematic separates operations; real collectives can overlap compute and run per unit or bucket.

The question for the profiler

How much of the additional parameter communication is exposed on the critical path?

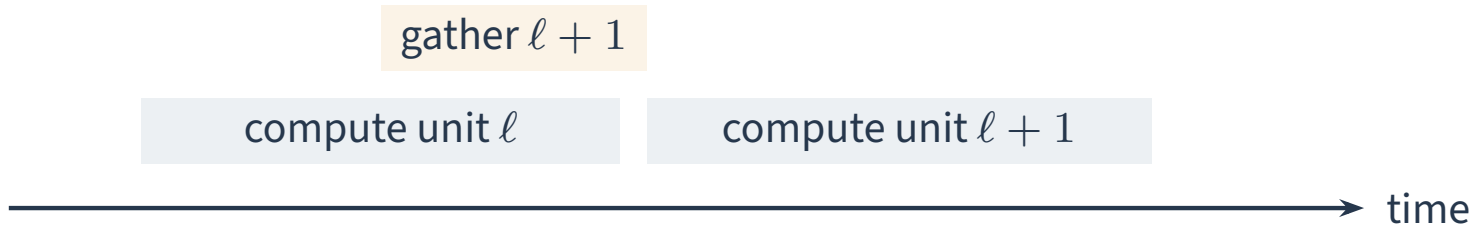
Communication volume and exposure are different

- **Volume**: total bytes transferred by all-gathers and reduce-scatters.
- **Exposure**: communication time that remains visible after overlap with compute.
- **Frequency**: number of collectives and their message sizes.

Why this matters

Two wrapping policies can move similar bytes but have very different step times because one creates many small latency-bound collectives or a longer exposed tail.

Prefetching tries to move the next unit early



Constraint

Prefetching can improve overlap but raises transient memory because current and next full units may coexist.

FSDP does not shard activation memory automatically

Full sharding reduces persistent parameter, gradient, and optimizer-state redundancy.

Still scales with batch and sequence

saved activations
attention workspaces
temporary full parameters
communication buffers

Combine FSDP with activation checkpointing when activations, not model state, dominate the peak.

CPU offload and sharding can work together

A second memory lever

Sharding distributes model state across ranks. Offloading places selected state in CPU memory; the two can be combined.

- Example: offload optimizer state and run the optimizer on CPU.
- Other policies offload parameters and gradients too; the exact behavior is implementation-specific.
- GPU–CPU traffic, CPU work and synchronization can reduce throughput. Measure the whole step.

Offload is a capacity tool, not a free speedup. See the website for API details.

Checkpoint: wrapping policy

Think, then compare

A run fits in memory but launches hundreds of tiny all-gathers and scales poorly. What wrapping change would you test, and which memory and timing quantities could regress?

PART 3

Choosing and comparing approaches

1. Where is the redundancy?
2. Execution and wrapping
3. Choosing and comparing approaches

A DDP–FSDP comparison needs a fixed contract

Match model, global token batch, optimizer, schedule, precision, data order, checkpoint cadence, and evaluation.

Correctness evidence

first-update comparison
loss trajectory
resume equivalence

Systems evidence

peak allocated memory
tokens per second
collective exposure
checkpoint time and size

From the idea to a PyTorch training loop

1. Initialize distributed ranks and select each device.
2. Construct the model; wrap Transformer blocks and the root as FSDP units.
3. Construct the optimizer on the sharded/wrapped model parameters.
4. Run forward, backward, optimizer step; measure memory and throughput.

The companion's `fsdp_utils.py` illustrates the FSDP1 wrapper and block policy. The lab is analytical; it does not execute this loop. PyTorch's newer FSDP2 uses `fully_shard`.

Choose FSDP when memory is the binding constraint

- Prefer DDP when the model fits comfortably and communication simplicity wins.
- Prefer FSDP when replicated state blocks model size, batch size, or context.
- Tune wrapping and prefetch from traces, not from a universal recipe.
- Treat checkpoint load and resharding as part of the production design.

Checkpoint: memory prediction

Think, then compare

Estimate idealized model-state memory per rank for P parameters, 16 bytes per parameter, and world size W. Then name three reasons the measured peak will be higher.

Key takeaways

1. ZeRO stages progressively shard optimizer state, gradients, and parameters.
2. FSDP materializes one wrapped unit through all-gather and returns sharded gradients.
3. Wrapping controls peak memory, collective size, frequency, and overlap.
4. Checkpoint format and a matched DDP baseline are part of the system claim.

PART 4

Optional: precision and checkpointing

---

4 Optional: precision and checkpointing

Mixed precision has three separate decisions

Quantity	Common choice	Main trade-off
parameter compute dtype	BF16	speed and temporary memory
gradient reduction dtype	BF16 or FP32	bandwidth versus numerical range
optimizer state dtype	FP32	stable moments at persistent cost

Experimental rule

When comparing DDP and FSDP, align precision policy. Otherwise, memory and numerical differences have multiple causes.

A distributed checkpoint is a data layout

Full

easy to consume
expensive to gather
rank 0 memory pressure

Sharded

scalable writes
world-size coupling
unless resharded

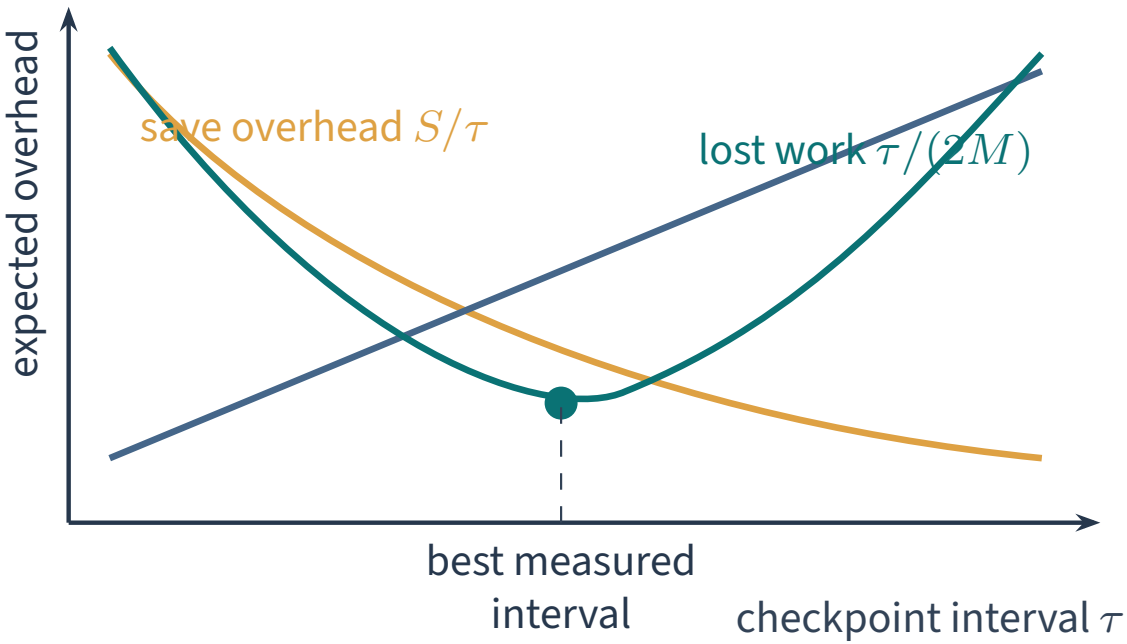
Distributed format

parallel I/O
metadata and
recovery complexity

Completion protocol

Publish a checkpoint only after every shard and its metadata are durable. A directory that exists is not proof of a complete save.

Checkpoint cadence and expected overhead



$$\text{rate} \approx \frac{S}{\tau} + \frac{\tau}{2M} + \frac{L}{M}$$

M: mean time between interruptions

S: exposed save time

L: recovery load time

Measure durable save completion and restart time; file creation alone does not define S or L.

Source: MVA Training and Deploying Large-Scale Models, Lecture 4 (2026)